

Leveraging Route Parameters



Lara Newsom

Software Technical Leader | Speaker | Angular GDE

[lara-newsom](#) | www.linkedin.com



Existing Implementation

header.component

```
// Menu links invoke the click handler
<a [routerLink]="PRODUCTS_ROUTE
  (click)="changeCategory('Seasonal Pies')
>Seasonal Pies</a>

// The method updates the pieService
changeCategory(category: Category){
  this.pieService.clearSelectedPie();
  this.pieService.setSelectedCategory(category);
}
```



Using the URL to Hold App State

The URL can hold user selections

These selections can be used
to perform operations in the code

Simplifies the code

Allows bookmarking user selections



Dynamic URL Parameter Types

**Required path
parameters**

**Optional matrix
parameters**

**Query
parameters**



Required Path Parameters

Dynamic segments declared in the path property of a route

Prefixed with a colon

The router treats path parameters like a wildcard

Path parameters in the URL look like every other path segment

The parameter is matched by its position in the path

Routes with static values in the same position must be declared first





Declaring Path Parameters

The string after the colon is the parameter name

Parameter names must be unique in the URL

Avoid using id for everything

`/products/:id/detail/:id`

Use specific names instead

`/products/:categoryId/detail/:productId`



Even though path parameters are visible in the URL, they are not accessible from everywhere.



Parameters cannot get off the bus before they have gotten on



Path parameters are only available to the component activated by the route and its children.

ActivatedRoute

A class exposed through the Angular router

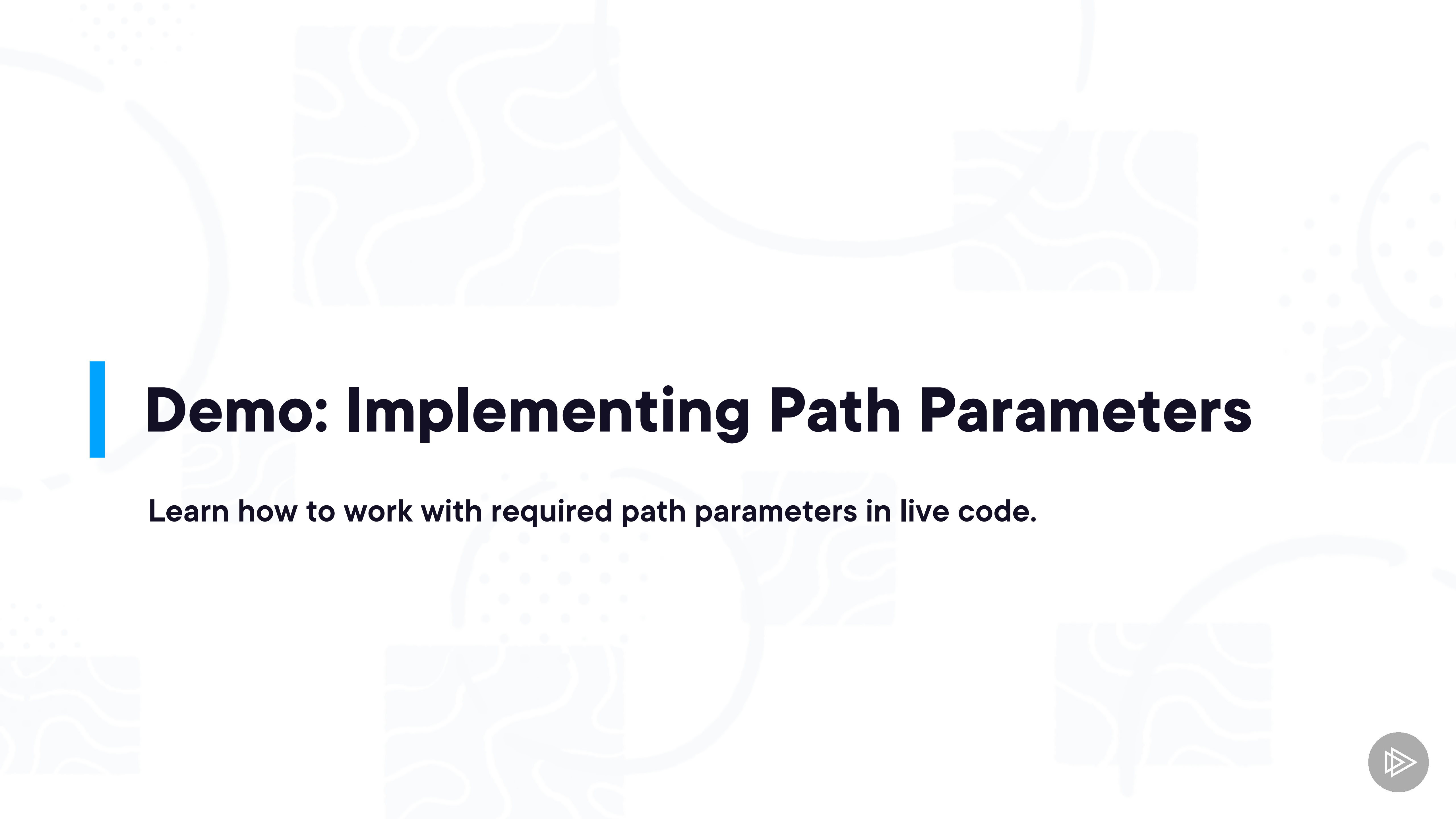
Exposes details about the current route

Some properties like snapshot are static and only provide the current value

Other properties, like paramMap and url, are observable streams that provide updated values if the route changes

The Angular docs list all properties
<https://angular.dev/api/router/ActivatedRoute>





Demo: Implementing Path Parameters

Learn how to work with required path parameters in live code.



Optional Matrix Parameters



Optional Matrix Parameters

Separated from path segments by a semicolon

Connected to specific path segments

Matrix URL notation has been around for decades



Matrix URL Notation

/shop /products; page=2; sort=asc

page

sort



Matrix URL Notation

/shop /products; page=2; sort=asc /cheesecake; selected=1

page

sort

selected





Matrix parameters can only be read by components activated by the route

Path parameters have the same limitation

Matrix parameters are removed from the URL when the user navigates off the route



Why Use Matrix Parameters?



Provide view-specific data like pagination

Matrix parameters are tightly scoped to their route

Avoids naming collisions and provides clear ownership



Implementing Matrix Parameters

```
// Pass matrix params like a path segment  
<a  
  [routerLink]="[ '/products', { page:1 } ]  
>Products</a>
```



Implementing Matrix Parameters

```
// Remove the path parameter from the route
{
  path: PRODUCTS_ROUTE,
  children: [...]
}

// Add the matrix param as a path segment
<a [routerLink]="
  [PRODUCTS_ROUTE, {categoryId:'Fruit Pies'}]"
>Fruit Pies</a>

// read the param in AllProductsComponent
// using withRouterComponentInputBinding
categoryId = input<Category>('All Pies');
```



Query Parameters



Query Parameters

Appended to the end of the path following a question mark

Additional parameters are separated by ampersands

Can be added to the URL using RouterLinks or the navigate method

QueryParamsHandling tells the router what should happen to existing query params in the URL



QueryParamsHandling

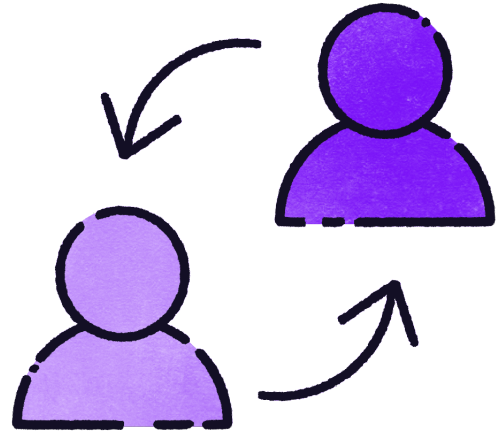
replace

merge

preserve



QueryParamsHandling



Replace handling replaces the existing query parameters with the incoming parameters



Merge handling combines new query parameters with the ones that already exist



Preserve handling keeps all existing query parameters and does not allow them to be added, deleted, or changed



Query Parameters

Query Parameters are globally available

Can be accessed by any class through the `ActivatedRoute`

Consuming classes do not have to be on the `ActivatedRoute`

Can pass data between unrelated classes in the component tree

Unique names help prevent name collisions



Accessing Query Parameters



Access using the `ActivatedRoute` in any class

**Exposed by two observable properties:
`queryParams` and `queryParamsMap`**

**Both emit the current query parameters when
the URL changes**



queryParamsMap

The get method reads a single param value

The getAll method reads multiple values of the same parameter



Demo: Implementing Query Parameters

Learn how to work with query parameters in live code.



Extra Practice Opportunities

Update the SideNavComponent to use a RouterLink instead of the navigate method

Don't forget to add the ariaCurrentWhenActive property

Update the BreadcrumbsComponent to handle the pield query param

Check your work in the final code for this video



Navigation Scrolling



withInMemoryScrolling

A RouterFeature that tells the router how to handle scrolling on navigation

Set the scrollPositionRestoration property to add navigation scrolling

The default is “disabled,” which does no scrolling

The “top” setting scrolls to the top of the page on each navigation

The “enabled” setting scrolls to the top on forward navigation and to the previous scroll position on backwards navigation

